LSDE COURSEWORK (EMATM0051)

Part 1 – ArtAI Architecture Design

Introduction

ArtAI is a simple web service where users upload an image and receive an AI-generated version of it. To make this work effortlessly for people in different parts of the world, the system needs to load fast, store images safely, and keep the AI model secure inside the cloud. It also has to alert us if the application gets more than 10,000 requests in a day. Based on these goals, we have to design a serverless AWS architecture that focuses on being easy to scale, cost-effective, and secure.

Architecture Overview

The overall design uses CloudFront and S3 for the website, API Gateway for the public API, Lambda for backend logic, and a private SageMaker endpoint for running the AI model. Uploaded and processed images are stored in separate S3 buckets, and DynamoDB keeps track of metadata. CloudWatch and SNS handle monitoring and daily alerts. The flow diagram (Figure 1) shows how each part connects to each other.

1. Frontend Delivery (CloudFront + S3 Buckets)

The UI (HTML/CSS/JS) is hosted as a static website in S3. CloudFront delivers this content via edge locations and enforces HTTPS which gives fast global loading and basic DDoS protection.

2. Amazon API Gateway

When the user uploads an image, the request is sent to Amazon API Gateway. This service acts as the main entry point into the backend. Then, it checks that the request is valid, applies rate limits if needed, and then forwards the request to the backend Lambda function. Using API Gateway keeps the external surface of the application tidy, safe and secure.

3. Backend Logic (AWS Lambda)

AWS Lambda handles all the backend steps. It receives the request from API Gateway and does these three things:

- **Stores the original image** in the S3 Uploads bucket
- **Calls the SageMaker endpoint** to run the AI model
- **Writes metadata** (paths, timestamps, status) to DynamoDB

Because, Lambda only runs when needed, this keeps the application cheap to operate and automatically scalable when traffic increases.

4. AI Model Inference (Amazon SageMaker – Private VPC)

The AI model runs on a private SageMaker endpoint inside a VPC so it cannot be accessed directly from the internet. Only Lambda is allowed to call it. SageMaker reads the original image from S3, applies the transformation, and uploads the processed image into the S3 Processed bucket. This setup gives us strong security while still allowing the model to scale when more users upload images.

5. Storage and Metadata

S3 Uploads Bucket: stores all the original images. Versioning will be enabled so that previous versions can also be recovered. Old versions will automatically move to Glacier to save the cost. Whenever a user uploads an image, S3 automatically stores it as a new version. This versioning acts as an instant backup, and older versions are moved to Glacier by lifecycle rules.

S3 Processed Bucket: stores the AI generated images. This bucket is also versioned.

DynamoDB: keeps metadata records such as image paths, timestamps and processing status.

6. Monitoring and Alerts (CloudWatch + SNS)

CloudWatch tracks how many requests hit the API and Lambda each day. A CloudWatch alarm is set to trigger when the system receives more than 10,000 requests in 24 hours. When this happens, the alarm sends a message to an SNS topic, which emails the administrator. This meets the alert requirement for the application.

7. Security and Network Design

IAM roles follow the least privilege option here. Both S3 and DynamoDB use encryption at rest. The SageMaker endpoint is completely private inside a VPC subnet, meaning the model is never exposed publicly. These small but important decisions help keep users' data and the model safe.

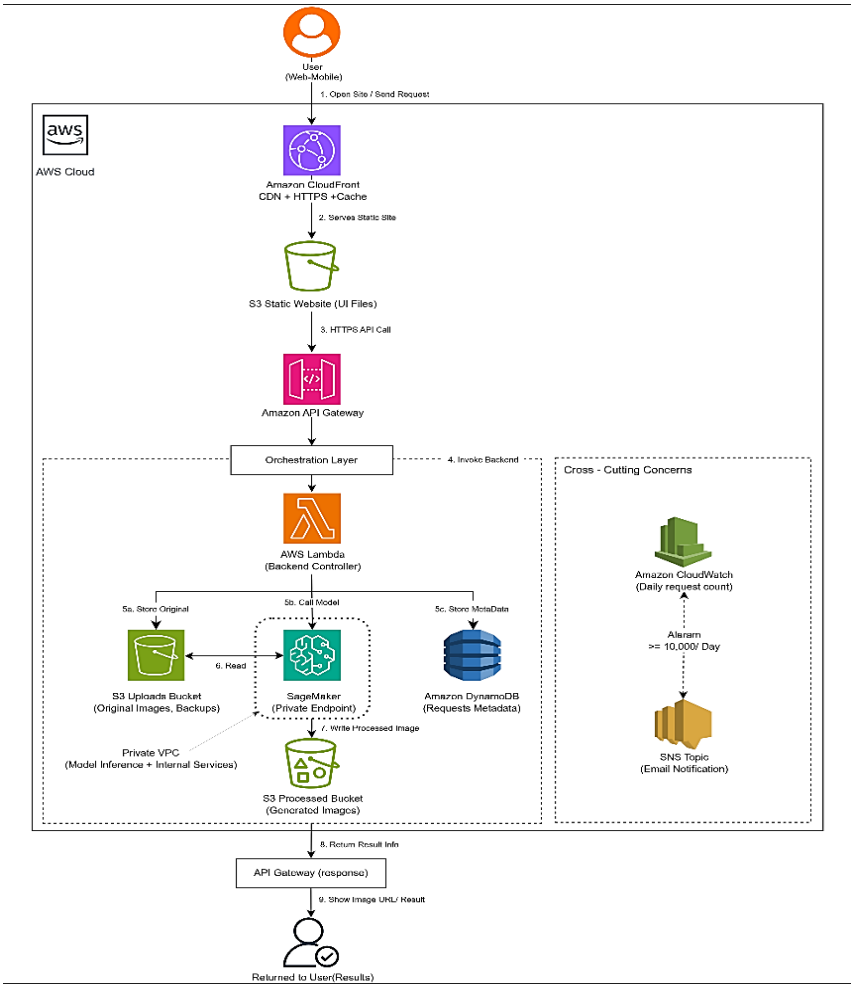


Fig. 1: End-to-End Architecture of the ArtAI Application

LSDE COURSEWORK (EMATM0051)

Part 2: Scaling the WordFreq Application

Introduction

Part 2 of this coursework primarily focuses on understanding, deploying, and improving the WordFreq application by using the services of AWS, introduced in the Cloud Foundations course. WordFreq is application that receives input through S3 buckets, distributes the work among SQS, processes each file among EC2 worker instances, and stores results in DynamoDB.

The intention of this part is to evaluate how application will behave under varied workloads, how it scales, and how its architecture can be strengthened using AWS best practices.

Task A forms a complete setup to verify that the entire end-to-end process is functioning as expected. After that, Task B enhances the system through the construction of auto-scaling for worker instances. Following that, Task C assesses performance during periods of high demand and simultaneously tests how efficient the scaling responses are. Then comes Task D, in which the design is modified in place to enhance reliability, data protection, cost reduction, and safety measures. Finally, Task E lists additional AWS tools that may boost scalability or reduce reliance on servers.

Task A: Installing and Understanding the WordFreq Application

Overview:

For Task A, the primary moto was to deploy the WordFreq application in the AWS Academy Learner Lab and make sure the basic end-to-end workflow was functioning.

I launched an Ubuntu-based EC2 instance, named wordfreq-dev. Attached the EMR_EC2_DefaultRole, which allows secure access to S3, SQS, and DynamoDB. I then connected via SSH and installed the needed packages, copying the coursework application folder onto the instance. Running the provided setup script created the DynamoDB table and prepared the worker environment.

I created the following AWS resources:

- **Two S3 Buckets:**
  an uploading bucket for raw text files and processing bucket for triggering the notifications when files are added
- **Two SQS queues:**
  wordfreq-jobs receives event messages from the processing bucket, and **wordfreq-results** stores the processed output.
- **DynamoDB table:**
  automatically created during setup, stores top 10 words for from each provided .txt file

The application workflow follows a simple pipeline **S3 (file added) → SQS (job message) → EC2 worker (processing) → DynamoDB + results queue**. When files are loaded into S3 buckets, a job message is sent to Amazon Simple Queue Service (SQS); EC2 Workers process the job message received and, while processing the file, retrieves the corresponding file and calculates the ten most frequently used words in the file; and the results of the processing will store the word frequency information in both DynamoDB and the Amazon SQS Results Queue.

To check the system, I uploaded a test text file and observed the full pipeline execution. Once the jobs queue picked up the message, the worker began processing it; afterward, DynamoDB saved the output.

Task B: Design and Implement Auto-scaling

Overview:

In this part of the coursework, the goal was to implement auto-scaling for the workers of WordFreq, so that system can automatically launch more EC2 instances when SQS jobs queue grows, and automatically scale down when the workload reduces. Apart from that, we also have to ensure that at least one worker will be always available. Due to which, I configured an Auto Scaling Group (ASG) from a launch template which I created from a pre-configured AMI and set up CloudWatch alarms to trigger scale-out and scale-in action based upon SQS queue length. Breakdown of each step is given below:

B 1. Creating the Worker AMI

I created the AMI from fully configured wordfreq-dev instance.

This AMI has:

- pre-installed WordFreq app
- correct IAM role
- S3/SQS/DynamoDB access
- systemd worker service running automatically

This worker AMI will provide consistent performance and backup for all future workloads.

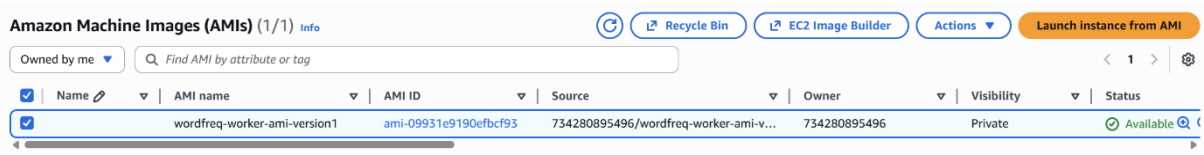


Fig.1: wordfreq-worker-ami-version1 (Available, Private)

B 2. Creating the Launch Template

In this step, I created the launch template by configuring following settings:

- AMI ID
- Instance type: **t3.micro**
- IAM instance profile: **EMR_EC2_DefaultRole**
- Security Group
- SSH key pair (PuTTY-compatible)

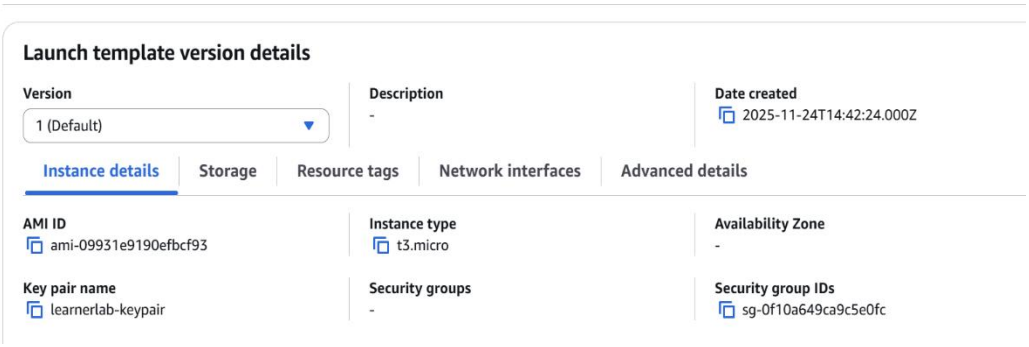


Fig.2: Launch Template Preview

B 3. Creating the Auto Scaling Group

I created an Auto-scaling group using the previously made launch template.

- Initial (default) configuration (to utilise minimum credits)
  Min = 0, Desired = 0, Max = 0
- Connected ASG to the same VPC/subnets as the dev instance, using EC2 health checks.
- Functional Auto-Scaling has
  Min = 1, Desired = 1, Max = 5 capacities



Fig.3: WordFreq Auto-scaling Group (Initial)

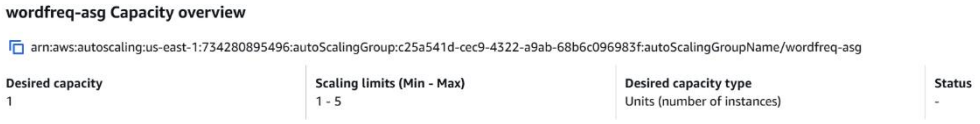


Fig.4: WordFreq Auto-scaling Group (testing phase)

B 4. Setting up CloudWatch Alarms

To control how the Auto Scaling group responds to changes in the workload, I set up two CloudWatch alarms based on the **ApproximateNumberOfMessagesVisible** metric from the wordfreq jobs.

Two alarms were set up:

1. **Scale-Out Alarm – Triggers when Workload Increases**

    - Trigger: queue length > **20 messages**
    - Evaluation: **2 datapoints / 2 minutes**
    - Purpose: launch new instance only when backlog is sustained.

2. **Scale-In Alarm – Triggers when Workload reduces**

    - Trigger: queue length ≤ **5 messages**
    - Evaluation: **5-minute window**
    - Purpose: avoid premature scale-in while workers finish active jobs.

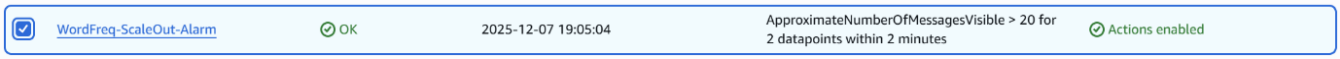


Fig. 5: Scale-Out Alarm (threshold > 20, 2 datapoints in 2 minutes)



Fig. 6: Scale-In Alarm (threshold ≤ 5, 1 datapoint within 5 minutes)

B 5. Defining Scaling Policies

I set up two scaling policies that dictate how your Auto Scaling Group will react once it is triggered by any of the CloudWatch alarms moving into the Alarm state.

1. **Scale-Out Policy**

    - Triggered by Scale-Out Alarm.
    - Step scaling is implemented
    - Increases desired capacity by **+1 instance**.
    - Included **120-second warm-up** so each new worker is fully ready before ASG re-evaluates.

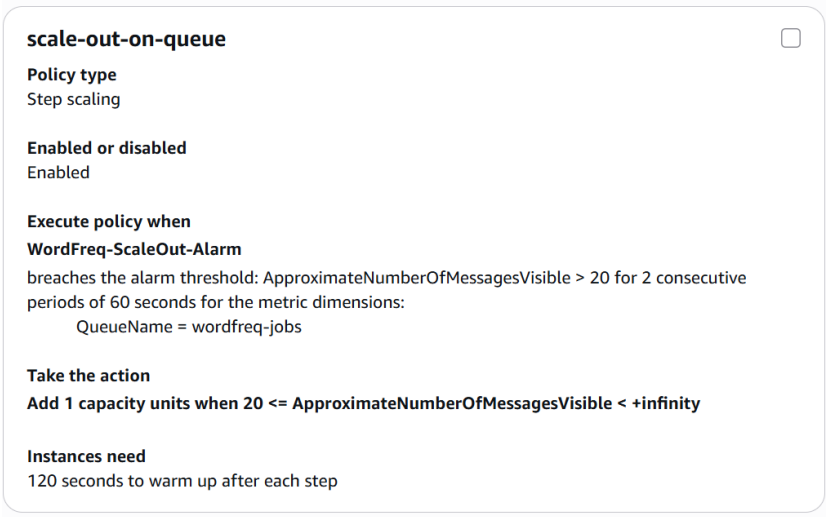


Fig. 7: Configured Scale-Out Policy

2. Scale-In Policy

- Triggered by Scale-In Alarm.
- Step scaling is implemented.
- reduces desired capacity by **-1 instance** (will remove/terminate newest instance first)
- Applied **180-second cooldown** to prevent early termination.
- Minimum capacity = **1**, ensuring continuous application availability.

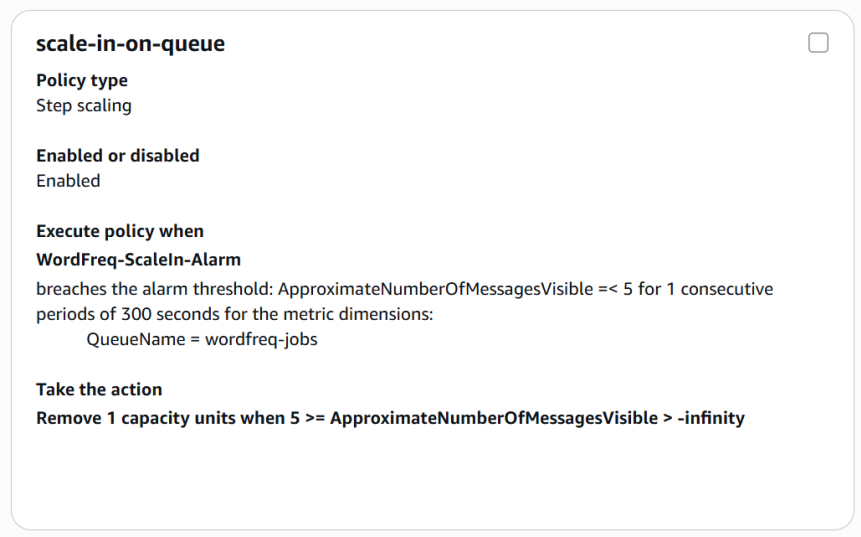


Fig. 8: Configured Scale-In Policy

B 6. Verifying the Auto-Scaling Configuration

Here, I ensured that all the components of auto-scaling group were correctly initialized.

- Auto-scaling group was linked to launch template
- both CloudWatch alarms were active
- Alarms were correctly associated with scaling policy

By doing this, I ensured that task C would be able to track the scaling behaviour, and not have to deal with any configuration issues.

Task C: Perform Load Testing

Overview:

The purpose of task C was to test whether the Auto-Scaling group was performed correctly to sudden increase in workload. This includes:

- preparation of 120 .txt files
- copying them into processing S3 bucket
- observing the behaviour of SQS, EC2 instances, DynamoDB Table
- recording processing times, and collecting screenshots that demonstrate how the system reacted under load.

C 1. Initializing

Before performing the actual loading test, I prepared the environment by:

- stopping the original wordfreq-dev EC2 instance (only ASG group instances would be used)
- reassured that 120 .txt files were present in uploadingS3 bucket
- AutoScaling group was enabled by setting to **min = 1, max = 5, desired = 1**

C 2. Performing the Loading Test

To initialise the loading test, all 120 .text files stored in processing bucket must be transferred to uploading bucket. To achieve this, I pasted the following command in SSH terminal (PUTTY):

aws s3 cp s3://zj-wordfreq-nov25-datauploading s3://zj-wordfreq-nov25-dataprocessing --exclude "*" --include "*.txt" --recursive

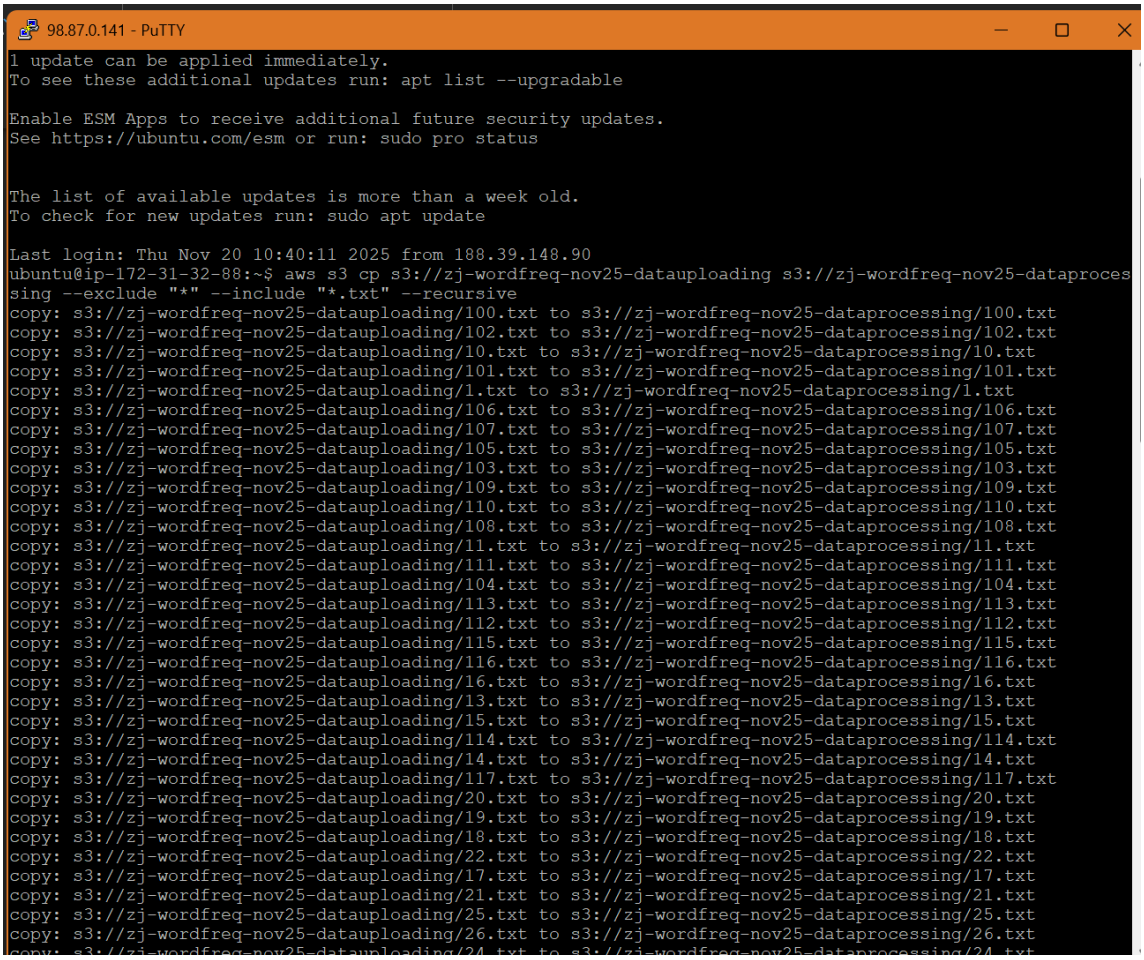
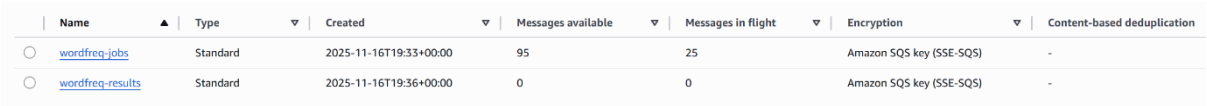


Fig. 9: Uploading files in S3 processing Bucket

(EC2 worker executing the aws s3 cp command to transfer all text files from the uploading bucket to the processing bucket for Task C load testing)

C 3. Observing the Auto-Scaling Behaviour

Once I started with the loading test, I was able to view that the Auto Scaling Group responded in a timely manner through monitoring the system through EC2, SQS, and CloudWatch by watching for updates. The first indication of scaling came from the SQS Jobs Queue, where it was obvious that there was a significant load on the queue, with the backlog reaching around 95 visible messages.



Name	Type	Created	Messages available	Messages in flight	Encryption	Content-based deduplication
wordfreq-jobs	Standard	2025-11-16T19:33+00:00	95	25	Amazon SQS key (SSE-SQS)	-
wordfreq-results	Standard	2025-11-16T19:36+00:00	0	0	Amazon SQS key (SSE-SQS)	-

Fig. 10: SQS jobs queue (95 messages visible)

As the queue began to grow, the AutoScaling group responded by launching additional worker instances time by time. This was visible in the EC2 console, where new t3.micro instances were born, showing as either “running” or “initializing”.

Name	Type	Created	Messages available	Messages in flight	Encryption	Content-based deduplication
wordfreq-jobs	Standard	2025-11-16T19:33+00:00	82	25	Amazon SQS key (SSE-SQS)	-
wordfreq-results	Standard	2025-11-16T19:36+00:00	13	0	Amazon SQS key (SSE-SQS)	-

Name	Type	Created	Messages available	Messages in flight	Encryption	Content-based deduplication
wordfreq-jobs	Standard	2025-11-16T19:33+00:00	23	38	Amazon SQS key (SSE-SQS)	-
wordfreq-results	Standard	2025-11-16T19:36+00:00	60	0	Amazon SQS key (SSE-SQS)	-

Name	Type	Created	Messages available	Messages in flight	Encryption	Content-based deduplication
wordfreq-jobs	Standard	2025-11-16T19:33+00:00	65	25	Amazon SQS key (SSE-SQS)	-
wordfreq-results	Standard	2025-11-16T19:36+00:00	30	0	Amazon SQS key (SSE-SQS)	-

Fig. 11, 12, 13: SQS jobs Queue (82, 65, 23 messages visible)

		i-0f7dba6a32151d159	Running	t3.micro	3/3 checks passed	View alarms +	us-east-1b	ec2-100-31-109-109.co…
		i-0f144df0e76a4b5cf	Running	t3.micro	3/3 checks passed	View alarms +	us-east-1b	ec2-54-147-9-16.comp…
		i-037d2620e1a8d01fd	Running	t3.micro	3/3 checks passed	View alarms +	us-east-1c	ec2-3-90-39-221.comp…
	dev-instance1	i-06e662ff4589a23ea	Running	t3.micro	3/3 checks passed	View alarms +	us-east-1a	ec2-44-203-184-28.co…
		i-0bde67c8f9fcf5533	Running	t3.micro	Initializing	View alarms +	us-east-1a	ec2-54-165-112-247.co…

Fig 14: EC2 console showing multiple t3.micro instances running / initializing

The spike occurred when the message count exceeded the configured threshold and returned to normal when additional workers came online. Once the workload was caught up to the processing capacity, the SQS jobs queue returned to zero visible messages, while the results queue contained 120 messages, which confirmed that all input files were processed and written to the output queue.

Name	Type	Created	Messages available	Messages in flight	Encryption	Content-based deduplication
wordfreq-jobs	Standard	2025-11-16T19:33+00:00	0	0	Amazon SQS key (SSE-SQS)	-
wordfreq-results	Standard	2025-11-16T19:36+00:00	120	0	Amazon SQS key (SSE-SQS)	-

Fig. 15: SQS jobs Queue (all 120 messages visible)



Fig 16: CloudWatch metric graph showing message spike above threshold (~90 messages)

Once the queue remained low for certain time, the scale-in alarm would be triggered and it will terminate excessive worker instances, reaching to minimum capacity of one EC2 instance. The following snapshot captured the EC2 activity logs, showing only one instance is running:

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone
wordfreq-dev	i-01cc5a4320190ee3e	Stopped	t2.micro	–	View alarms	us-east-1b
	i-0f98b47a43bd5ffc0	Terminated	t3.micro	–	View alarms	us-east-1c
	i-0f7dba6a32151d159	Terminated	t3.micro	–	View alarms	us-east-1b
	i-0c3a8e1afe0955adc	Running	t3.micro	3/3 checks passed	View alarms	us-east-1a
dev-instance1	i-01f163ff57f2d541f	Terminated	t3.micro	–	View alarms	us-east-1b

Fig 17: EC2 console showing only one EC2 instance running

C 4. Optimizing the Auto-Scaling Behaviour

After the completion of the initial loading test, it was time to adjust some Auto-Scaling features and CloudWatch alarms to improve the application's response time and overall performance. Optimisation was the best when I adjusted my wordfreq application as follows:

- reduced CloudWatch alarm period to 10 seconds
- lowering the datapoints to 1 out of 1

Due to which, the execution time of application was reduced from 9 minutes to approx. 4 minutes after optimisation.



Fig 18: CloudWatch metric showing the optimised scaling response for optimised load testing

(The scale-out alarm triggers within 10 seconds as the queue exceeds the threshold, and the backlog is cleared rapidly as additional workers are launched)

Parameter	Base (Default Settings)	Optimized Test Settings
Instance type used	t3.micro	t3.small
Scale out threshold	>20 messages	>20 messages
Scale out period	60 seconds	10 seconds
Scale-out datapoints	2 out of 2	1 out of 1
instance warm-up	120 seconds	120 seconds
Scale-in threshold	≤ 5 messages	≤ 5 messages
Scale-in period	60 seconds	10 seconds
Scale-in datapoints	1 datapoint over 300 seconds	1 datapoint (10 sec)
Scale-in cooldown	180 seconds	180 seconds
ASG capacity during test	min=1, desired=1, max=5	min=1, desired=1, max=5
Reaction speed	Slow (1–2 min delay)	Immediate (≤10 seconds)
Total processing time (120 files)	**8–9 minutes**	**4-5 minutes**

Table 1: Comparison of Default vs Optimised Auto-Scaling Settings

C 5. Conclusion:

As a result of load testing, we found that the wordfreq app scales well when suitable auto-scaling methods are applied - this means instances adjust dynamically based on demand. Following minor improvements, speed increased noticeably, cutting overall processing duration close to 50%. In general, the scaling system worked correctly, showing reliable reactions, solid efficiency, along with lower resource costs.

Task D: Optimising the WordFreq Architecture

Overview:

The main focus of task D was to improve the existing wordfreq application with respect overall reliability, security and cost efficiency using what we learnt in AWS course. So, these re-structured preserves the existing flow of SQS → EC2 → DynamoDB workflow, but will strengthen each aspect of the application which was lacked in previous architecture.

D 1. Increasing Resilience and Availability

The load testing, performed in task C gives us the input that the application experienced some backlogs during sudden thresholds. So, to overcome this query, this time, the worker EC2 instances (both existing and new born) are distributed over an Auto Scaling group deployed across three different subnets. Both S3 buckets have versioning ON. It means, these buckets are protected against accidental overwrites and failure (or deletion).

D 2. Long Term Backup and Data Protection

The application's input plus processed files are useful data sets. So, the Uploading S3 Bucket moves older ones to Glacier Deep Archive through a Lifecycle Rule - keeps them safe while cutting costs. Meanwhile, the DynamoDB table runs Point-in-Time Recovery, meaning it can roll back to any moment within 35 days. These updates keep the system fixable, just in case something gets messed up by mistake.

D 3. Cost-Efficient Processing for Burst Workloads

During initial testing, task C demonstrated volatile behaviour, sometimes processing number of messages with intense, then short and sometimes steady. Also, to avoid extra credits, auto-scaling was setup to minimum capacity of 0 to 1. To add that, additional worker instances can be launched using EC2 spot instances to reduce cost significantly and not hand over entire application's load to one instance only.

DynamoDB's on demand capacity mode will save significant amount of cost, eliminating the foresight of to set capacities.

D 4. Improving Security and Access Control

Security aspect can be revaluated using **IAM least privilege roles**, which restrict workers to utilise specific S3 buckets, SQS queues and DynamoDB tables for processing. EC2 instances run within a VPC while security group allow SSH from verified Ips alone. For audits, CloudTrail logs every API request using continuous monitoring.

Overall, we can conclude that this updated design is more reliable, considering the reduction in cost when the application is not active and overall information is secured using version control and point in time recovery (PITR). The system remains simple and fully aligned with AWS cloud foundation principles.

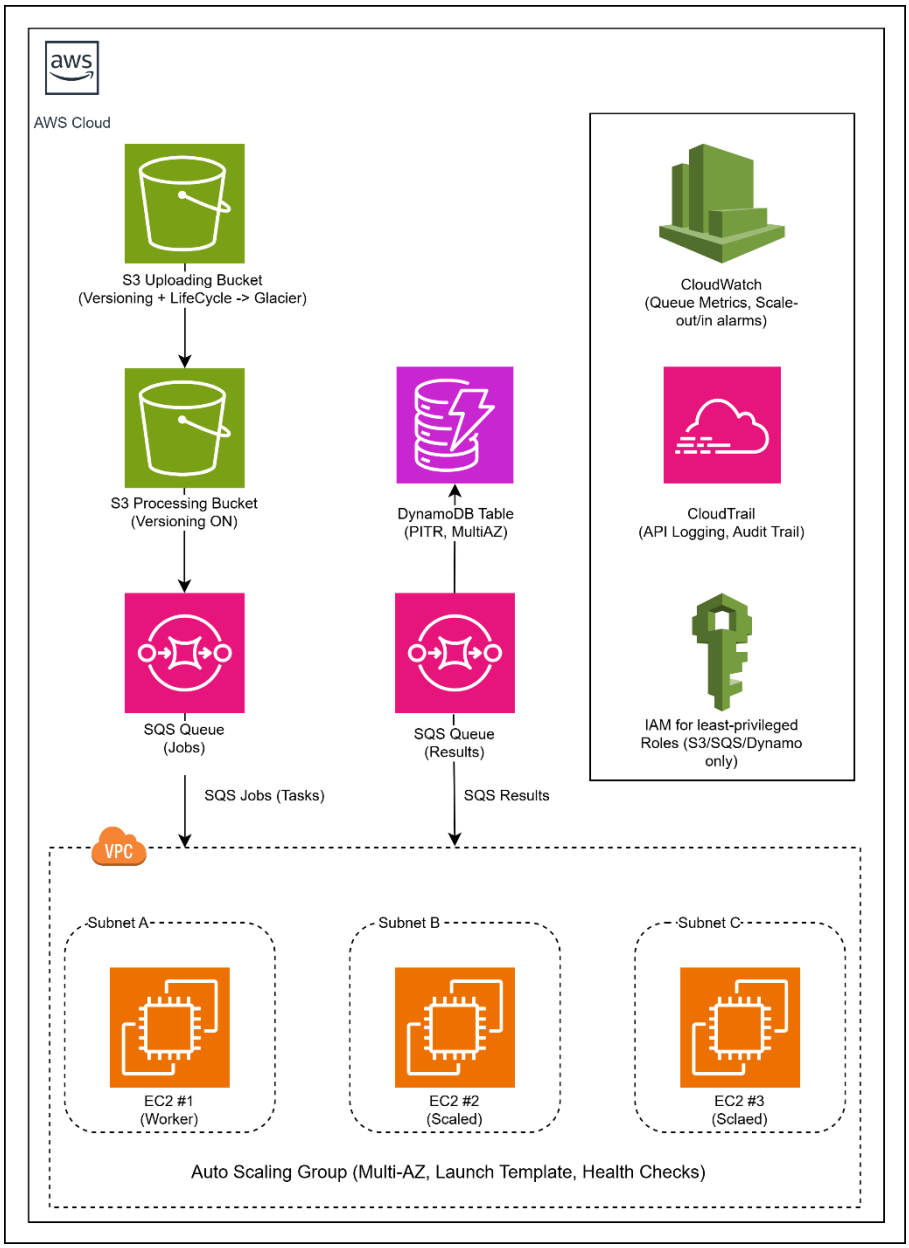


Fig. 19: Improved WordFreq Architecture

Task E: Further Improvements

Beyond the EC2 Auto Scaling architecture used in WordFreq, there are several AWS services that can process large numbers of text files more efficiently, with higher scalability and reduced operational overhead. Two alternatives that naturally extend the WordFreq design are AWS Lambda and AWS Batch.

E 1. AWS Lambda + S3 + SQS

A fully serverless environment would be capable of replacing the use of EC2-based workers. As soon as a file is uploaded to the S3 bucket used for processing, a Lambda function would be triggered directly. Compared with EC2-based processing where startups may be delayed, a Lambda function grows extremely fast. This function serves thousands of tasks within mere seconds.

A Lambda function would be ideal for light to medium use in processing text because one only gets charged when it is running. This means that bursts of usage will mean little extra cost because one only gets charged when running. As opposed to previous setups where outputs were fed directly to DynamoDB, outputs go to this database once a job is completed. The entire system does not require a server but requires little management.

E 2. AWS Batch with EC2 Compute Environments

AWS Batch offers a sophisticated way of batch processing that can be used for tasks where better compute resource management capabilities are a prerequisite. Unlike managing an Auto Scaling Group manually, AWS Batch starts and stops EC2 instances based on jobs being submitted to its queue. While traditional setups require constant observation for optimization purposes, this service adjusts compute resources automatically with any changes in workload. Resource optimization occurs with ease based on changes in workload without any user intervention. As a consequence of a drop in workload levels being submitted for processing, scaling down also occurs.

A separate batch task for each input text file is generated. The WordFreq worker can be packaged into a container and submitted as a job template. This allows multiple containers running EC2-based jobs to be run in parallel. This allows for automatic retries, error processing, priority sorting, and smart resource assignments.

Altogether, this approach makes manual scaling configuration a thing of the past while offering superior performance for large data without compromising insight into machine selection and cost.

[Account credentials removed from public copy]

References:

[1] Amazon Web Services, *Amazon S3 Developer Guide*.
https://docs.aws.amazon.com/AmazonS3/latest/dev/Welcome.html

[2] Amazon Web Services, *Amazon EC2 Auto Scaling User Guide*.
https://docs.aws.amazon.com/autoscaling/ec2/userguide/

[3] Amazon Web Services, *Amazon Simple Queue Service (SQS) Developer Guide*.
https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/

[4] Amazon Web Services, *Amazon DynamoDB Developer Guide*.
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/

[5] Amazon Web Services, *Amazon CloudWatch Monitoring Guide*.
https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/

[6] Amazon Web Services, *AWS Lambda Developer Guide*.
https://docs.aws.amazon.com/lambda/latest/dg/

[7] Amazon Web Services, *Amazon SageMaker Developer Guide*.
https://docs.aws.amazon.com/sagemaker/latest/dg/

[8] Amazon Web Services, *Amazon Simple Notification Service (SNS) Developer Guide*.
https://docs.aws.amazon.com/sns/latest/dg/

[9] Amazon Web Services, *AWS Well-Architected Framework*. Available at:
https://docs.aws.amazon.com/wellarchitected/latest/framework/

[10] Amazon Web Services, *AWS Identity and Access Management (IAM) Best Practices*.
https://docs.aws.amazon.com/IAM/latest/UserGuide/best-practices.html

[11] Amazon Web Services, *AWS CloudTrail User Guide*.
https://docs.aws.amazon.com/awscloudtrail/latest/userguide/

[12] Amazon Web Services, *Auto Scaling Step and Simple Scaling Policies*.
https://docs.aws.amazon.com/autoscaling/ec2/userguide/as-scaling-simple-step.html

[13] Amazon Web Services, *Amazon S3 Lifecycle Configuration*.
https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lifecycle-mgmt.html

[14] Amazon Web Services, *Point-in-Time Recovery (PITR) for Amazon DynamoDB*.
https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/PointInTimeRecovery.html

[15] Kleppmann, M., *Designing Data-Intensive Applications*. O'Reilly Media, 2017.